

Object-Oriented Programming

Classes, Objects & the Four Pillars — Encapsulation, Abstraction, Inheritance, Polymorphism

Session 10

What We're Covering Today

- Why OOP exists — the shift from separate data and functions to bundled objects
- Classes and objects — __init__, self, attributes, and methods
- Encapsulation — controlling access with naming conventions and @property
- Abstraction — hiding complexity behind a simple interface
- Inheritance — building new classes from existing ones, and super()
- Polymorphism — one interface, many behaviors
- Python-specific details — dunder methods, classmethod/staticmethod, MRO
- Today's tasks and two mini projects combining everything



PART 1

Why Object-Oriented Programming?

Every program you've written so far kept its data and its logic separate. OOP bundles them together.

What Is OOP, Really?



Data + behavior, bundled

- Object-oriented programming organizes code around objects — self-contained units that bundle data (attributes) together with the functions that act on that data (methods)
- Up to now, a "dog" in your code was just a dictionary, and bark() was a separate function you had to remember to call correctly, on the right dictionary
- In OOP, a Dog object carries its own data and its own behavior together — dog.bark() instead of bark(dog_dict)
- This isn’t a new language feature — it’s a different way of organizing the same logic, and it’s how almost every large, real-world codebase is structured

Procedural vs. Object-Oriented — The Same Problem, Two Ways

What You've Been Doing

- A dictionary holds the data: dog = {"name": "Rex", "energy": 100}
- A separate function operates on it: def bark(dog): …
- Nothing stops another part of the code from passing the wrong dictionary into bark()
- As the program grows, data and the functions that use it drift further apart

The OOP Way

- A class defines both the data and the behavior together, in one place
- rex = Dog("Rex", 100) creates an object that owns its own data
- rex.bark() can only ever act on rex's own data — there's no dictionary to mix up
- Data and behavior stay physically next to each other as the program grows

Where You Already See This in the Real World



A Car

Its data is speed, fuel level, gear. Its behavior is accelerate(), brake(), refuel() — all bundled into one "car" concept



A Bank Account

Its data is balance and owner. Its behavior is deposit() and withdraw() — the only way the balance is allowed to change



A Game Character

Its data is health, position, inventory. Its behavior is attack(), move(), heal() — every game engine is built this way



PART 2

Classes & Objects

A class is the blueprint. An object is one specific thing built from that blueprint.

What Is a Class?



The blueprint, not the thing

- A class is a blueprint — it defines what data and behavior every object built from it will have
- A blueprint for a house isn’t a house you can live in — a class isn’t something you use directly either
- An object (also called an instance) is one actual thing built from that blueprint — with its own real values filled in
- You can build as many objects as you want from one class, and each one is independent of the others

Creating a Class and an Object

- class Dog: starts a class definition — by convention, class names are CapitalizedLikeThis
- Dog() calls the class like a function to create a new object — this is called instantiation
- type() confirms what you already know — rex really is a Dog

```
class Dog:
    pass    # empty class for now

rex = Dog()     # create an object (instantiate)
milo = Dog()    # a second, independent object

print(type(rex))    # <class '__main__.Dog'>
print(rex is milo)  # False — two separate objects
```

__init__ — The Constructor

- __init__ is a special method that runs automatically every time a new object is created
- It’s where you set up the object’s starting data — "construct" its initial state
- The double underscores mark it as a dunder method — Python-reserved, called automatically, never called directly. "Dunder" is short for "double underscore"

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

rex = Dog("Rex", "Labrador")
print(rex.name)     # Rex
print(rex.breed)    # Labrador
```

self — What It Actually Means

- self refers to the specific object a method is being called on — "this particular dog," not dogs in general
- Every instance method needs self as its first parameter — Python passes it automatically, you never pass it yourself
- self.name = name inside __init__ literally means "store this value on this specific object"
- The name "self" is just a convention (like "args") — what matters is its position as the first parameter

```
class Dog:
    def __init__(self, name):
        self.name = name    # attach to THIS object

    def bark(self):
        print(f"{self.name} says Woof!")

rex = Dog("Rex")
rex.bark()    # Python secretly calls Dog.bark(rex)
# Rex says Woof!
```

A Class Is a Blueprint — Objects Are the Real Things

```
class Dog:

    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says Woof!")
```

THE BLUEPRINT — DEFINED ONCE

rex = Dog(...)

name = "Rex" · breed = "Labrador"

milo = Dog(...)

name = "Milo" · breed = "Beagle"

buddy = Dog(...)

name = "Buddy" · breed = "Poodle"

One class, three completely independent objects — each carries its own data, but shares the same bark() method.

Instance Attributes vs. Class Attributes

Instance Attributes

- Set inside __init__ using self.name = ...
- Belong to one specific object — each object has its own separate copy
- This is what you'll use for almost all object data — name, balance, position

Class Attributes

- Defined directly inside the class body, outside any method
- Shared by every object built from that class — one copy, not one per object
- Good for constants or defaults every instance should share, like species = "Canine"

Practice Set — Classes & Objects

Try these yourself first — no code shown, that's the point

1. Write a Car class with __init__ that sets make, model, and year

2. Add a method describe(self) that prints a sentence using all three attributes

3. Create two Car objects with different data, and call describe() on each

4. Add a class attribute wheels = 4 shared by every car, and print it from both objects



PART 3

The Four Pillars of OOP

Every object-oriented language, including Python, is built around these four ideas

The Four Pillars, at a Glance



Encapsulation

Bundling data with the methods that protect it, and controlling how outside code can access it



Abstraction

Hiding complex internal details behind a simple, easy-to-use interface



Inheritance

Building new classes on top of existing ones, reusing and extending their behavior



Polymorphism

Different classes responding to the same method call in their own appropriate way

Pillar 1: Encapsulation



Protect the data inside

- Encapsulation means bundling data with the methods that operate on it, and restricting direct outside access to that data
- A bank account shouldn’t let any code just set balance = -500 or balance = +500 directly — changes should only happen through controlled methods like withdraw(), deposit()
- This protects an object from being put into an invalid or inconsistent state by code elsewhere in the program
- Python doesn’t enforce this with hard restrictions the way some languages do — it relies on naming conventions and trust, which is a deliberate design choice ("we’re all consenting adults here")

Public, Protected, and Private — By Convention

- name is public — freely accessible from anywhere, the normal default
- _balance (single underscore) is a convention meaning "internal, please don’t touch directly" — Python doesn’t block it, it just signals intent
- __pin (double underscore) triggers name mangling — Python renames it internally, making accidental outside access genuinely difficult

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner       # public
        self._balance = balance   # "internal" by convention
        self.__pin = "4471"      # name-mangled, hard to reach

acc = Account("  Ahmed", 1000)
print(acc.owner)        # "Ahmed" — fine, it's public
print(acc._balance)      # works, but signals "you shouldn't"
print(acc.__pin)        # AttributeError — mangled to _Account__pin
```

@property — Controlled Access, the Python Way

- @property turns a method into something that reads like a plain attribute — balance instead of get_balance()
- A matching @balance.setter lets you validate any value before it's allowed to be stored
- This is the real, idiomatic way encapsulation is done in Python — clean access on the outside, control on the inside

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, amount):
        if amount < 0:
            raise ValueError("Balance can't be negative")
        self._balance = amount

acc = Account(1000)
acc.balance = 500      # runs the setter, validated
acc.balance = -50      # ValueError — blocked
```

Practice Set — Encapsulation

Try these yourself first — no code shown, that's the point

1. Write a Wallet class with a protected _cash attribute and add_cash()/spend_cash() methods

2. Make spend_cash() refuse to let _cash go below zero

3. Turn balance into a @property with a setter that rejects negative values

Pillar 2: Abstraction



Hide the how, show the what

- Abstraction means hiding complicated internal details behind a simple interface — the user only needs to know what a method does, not how it is done
- You call car.start_engine() without knowing anything about fuel injection or ignition timing — the complexity is hidden behind a simple method name
- In Python, this often shows up as an abstract base class — a class that defines what methods a subclass must have, without saying how they work
- Abstraction and encapsulation work together, but they solve different problems: encapsulation protects data, abstraction simplifies interfaces

Abstract Base Classes — Defining a Required Interface

- ABC and @abstractmethod (from the abc module) let you define a class that can’t be used directly — only inherited from
- Any subclass is forced to implement every abstract method, or Python refuses to let you create it
- This guarantees every "shape" in a program has an area() method, without dictating how each one calculates it

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        print("Woof!")

class Cat(Animal):
    def make_sound(self):
        print("Meow!")

# You cannot create an object of an abstract class
animal = Animal()   # TypeError
```

Practice Set — Abstraction

Try these yourself first — no code shown, that's the point

1. Write an abstract Vehicle class with an abstract method fuel_type()

2. Create two subclasses, ElectricCar and PetrolCar, each implementing fuel_type() differently

3. Try instantiating Vehicle directly and confirm Python blocks it

Pillar 3: Inheritance



Build on what already exists

- Inheritance lets a new class (the child, or subclass) reuse and extend the data and behavior of an existing class (the parent, or superclass)
- A GoldenRetriever is a Dog — it automatically gets everything a Dog has, and can add its own extra behavior on top
- This avoids rewriting the same code in every related class — shared behavior lives once, in the parent
- It also models real-world "is-a" relationships directly in code: a Car is a Vehicle, a Manager is an Employee

Basic Inheritance

- class Cat(Animal): means Cat inherits everything Animal has — Animal goes in the parentheses
- Cat automatically gets Animal's __init__ and eat() without rewriting them
- isinstance() confirms a Cat is also considered an Animal — the inheritance relationship is real, not just naming

```
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating")

class Cat(Animal):     # Cat inherits from Animal
    pass

whiskers = Cat("Whiskers")
whiskers.eat()              # inherited — works with no rewrite
isinstance(whiskers, Animal)    # True
```

super() — Calling the Parent's Version

- When a subclass defines its own __init__, it replaces the parent's __init__ entirely — unless you explicitly call it
- super().__init__(...) runs the parent's constructor first, so you don't have to duplicate its logic
- This pattern — extend, don't replace — is the standard way subclasses build on their parent

```
class Animal:
    def __init__(self, name):
        self.name = name

class Cat(Animal):
    def __init__(self, name, indoor):
        super().__init__(name)   # reuse the parent's setup
        self.indoor = indoor     # then add Cat-specific data

whiskers = Cat("Whiskers", True)
whiskers.name     # "Whiskers" — handled by Animal
whiskers.indoor   # True — handled by Cat
```

Overriding Methods

- A subclass can redefine a method it inherited, giving it different, more specific behavior — this is called overriding
- Python always uses the closest, most specific version of a method it can find on the object's own class first
- The parent's original version is still there if you need it — call it explicitly with super().speak()

```
class Animal:
    def speak(self):
        print("Some generic animal sound")

class Dog(Animal):
    def speak(self):      # overrides Animal.speak
        print("Woof!")

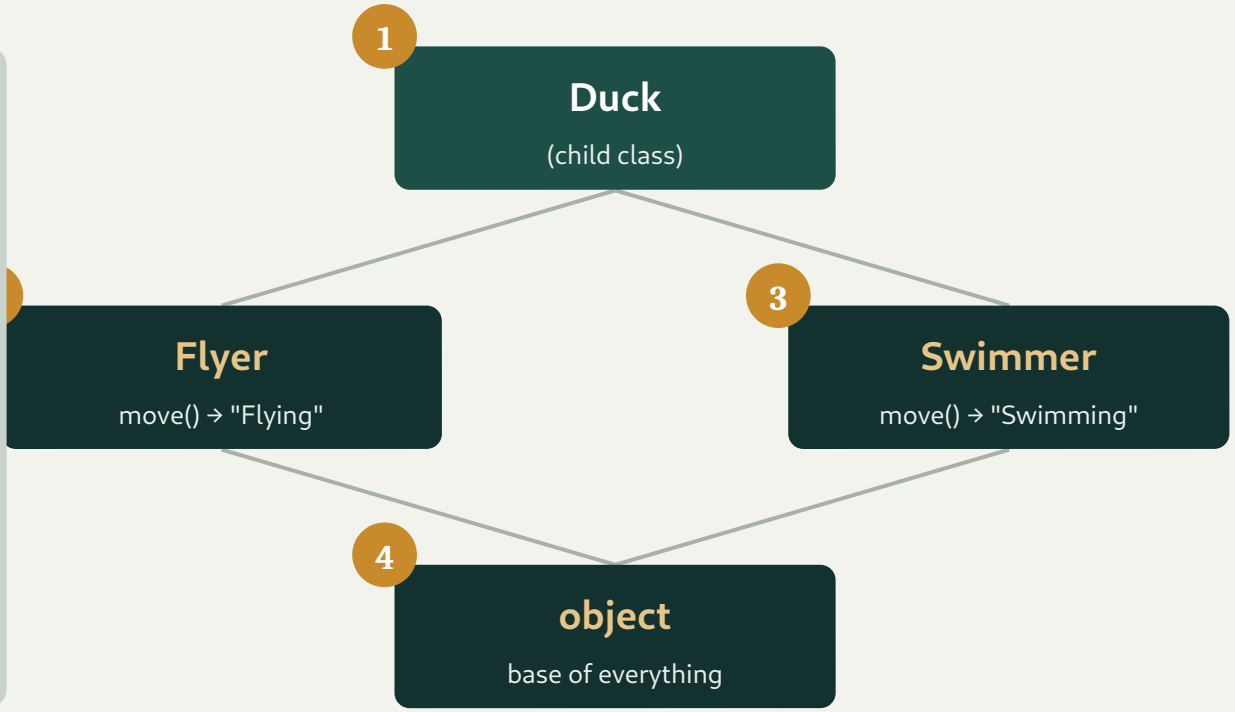
Animal().speak()   # Some generic animal sound
Dog().speak()      # Woof!
```

Method Resolution Order — Who Answers the Call?

Duck.__mro__

1. Duck
2. Flyer
3. Swimmer
4. object

Python searches left to right, so Flyer's move() wins over Swimmer's.



Multiple Inheritance — Inheriting From More Than One Class

- Python allows a class to inherit from more than one parent at once — class Duck(Flyer, Swimmer)
- If more than one parent defines the same method, Python resolves the conflict using its Method Resolution Order (MRO)
- You can inspect the exact search order yourself with ClassName.__mro__ — useful whenever behavior seems ambiguous

```
class Flyer:
    def move(self):
        print("Flying")

class Swimmer:
    def move(self):
        print("Swimming")

class Duck(Flyer, Swimmer):
    pass

Duck().move()        # Flying — Flyer is checked first
print(Duck.__mro__)  # shows the full search order
```

Practice Set — Inheritance

Try these yourself first — no code shown, that's the point

1. Write an Employee class with name and salary, then a Manager subclass adding a team_size attribute

2. Use super().__init__() inside Manager rather than repeating Employee's setup logic

3. Override a raise_salary() method in Manager so managers get double the normal raise percentage

4. Print MyClass.__mro__ for a class with two parents, and explain the order out loud before checking

Pillar 4: Polymorphism



Same call, different behavior

- Polymorphism means different classes can respond to the exact same method call, each in its own appropriate way
- "Poly" (many) + "morph" (forms) — one interface, many shapes of behavior underneath it
- You already saw the mechanism in the last part: method overriding is exactly how polymorphism happens in practice
- The payoff is code that works on any object with the right method, without needing to know or check its exact class first

Polymorphism in Action — One Loop, Many Behaviors

- Every shape in the list has an area() method — but each class calculates it completely differently
- The loop doesn't check what type of shape it has — it just trusts that .area() exists and calls it
- This is the real payoff of polymorphism: code that's written once and works correctly for classes that don't exist yet

```
class Circle:
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2 # we have the same method but it is used in a different manner for both classesssssss

class Square:
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2 # we have the same method but it is used in a different manner for both classes

shapes = [Circle(3), Square(4)]
for shape in shapes:
    print(shape.area())   # each uses its OWN area() logic
# 28.27...
# 16
```

Duck Typing — Python’s Real Polymorphism

- "If it walks like a duck and quacks like a duck, it’s a duck" — Python doesn’t check an object’s class, only whether it has the method being called
- Two totally unrelated classes can both work with the same code, as long as they both implement the method it expects
- This is looser and more flexible than polymorphism in many other languages — it’s driven entirely by behavior, not by shared inheritance

```
class Duck:
    def sound(self): print("Quack!")

class Dog:
    def sound(self): print("Woof!")

def make_it_talk(animal):
    animal.sound()   # no type check — just calls .sound()

make_it_talk(Duck())   # Quack!
make_it_talk(Dog())    # Woof! — completely unrelated classes, both work
```

Practice Set — Polymorphism

Try these yourself first — no code shown, that's the point

1. Write three classes (Cat, Dog, Cow) each with their own sound() method

2. Loop over a list containing one of each and call .sound() on every item without checking type

3. Write a function that accepts any object with a .describe() method, and call it with two unrelated classes to prove duck typing works



PART 4

Python-Specific OOP Details

Beyond the four pillars — the conventions and tools every real Python class actually uses

__str__ vs. __repr__

- By default, printing an object shows an unhelpful memory address — <__main__.Dog object at 0x...>
- __str__ defines what print() and str() show — aimed at humans, readable
- __repr__ defines what appears in the shell/debugger, and is used as a fallback if __str__ is missing — aimed at developers, ideally unambiguous

```
class Dog:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"{self.name}, a very good dog"

    def __repr__(self):
        return f"Dog(name='{self.name}')"

rex = Dog("Rex")
print(rex)   # Rex, a very good dog   (__str__)
rex          # Dog(name='Rex')        (__repr__, in the shell)
```

More Dunder Methods — Operator Overloading

- Dunder methods let your own objects work with Python's built-in operators and functions — +, ==, len(), and more
- __eq__ defines what == means for your objects — by default, Python only checks if they're the exact same object in memory
- __add__ defines what + does — this is exactly how "vector1 + vector2" works for custom classes

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __len__(self):
        return int((self.x**2 + self.y**2) ** 0.5)

Point(1, 2) == Point(1, 2)   # True
Point(1, 2) + Point(3, 4)    # Point object with x=4, y=6
```

@classmethod vs. @staticmethod

@classmethod

- Receives the class itself (cls) instead of an instance (self) as its first argument
- Can access and modify class-level state, and is commonly used to build alternate constructors
- Called on the class: Dog.from_string(...)

@staticmethod

- Receives neither self nor cls — it’s just a regular function that happens to live inside the class
- Used when logic is related to the class conceptually, but doesn’t need any instance or class data
- Called the same way: Dog.is_valid_name("Rex")

@classmethod as an Alternate Constructor

- This is the single most common real-world use of @classmethod — offering more than one way to build an object
- from_string builds a Dog from one format (a comma-separated string) while __init__ handles the normal way
- Both ultimately create the same kind of object — cls(...) inside a classmethod calls the real constructor

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data):
        name, age = data.split(",")
        return cls(name, int(age))    # calls Dog(name, age)

rex = Dog.from_string("Rex,3")
rex.name, rex.age    # ('Rex', 3)
```

"Everything Is an Object" in Python



Not just your own classes

- Every value in Python — numbers, strings, lists, functions, even classes themselves — is an object of some class
- (5).bit_length() works because even an int is an object with methods — type(5) really is <class 'int'>
- This is why dir() and type() work uniformly on absolutely anything in Python — there’s no special-cased "primitive" data
- Understanding your own classes makes Python’s built-in types make more sense too — they’re built the same way yours are

A Famous Python Gotcha: Mutable Class Attributes

- A mutable class attribute (like a list) is shared by every instance — exactly like any other class attribute
- Appending to it from one object silently affects every other object, because there's only ever one list
- The fix: create mutable attributes inside __init__ with self, so each object gets its own independent copy

```
# WRONG — shared across every instance
class Dog:
    tricks = []        # class attribute — ONE list for all dogs
    def add_trick(self, trick):
        self.tricks.append(trick)

rex = Dog(); milo = Dog()
rex.add_trick("sit")
milo.tricks    # ['sit'] — leaked into milo too!

# RIGHT — each instance gets its own list
class Dog:
    def __init__(self):
        self.tricks = []   # instance attribute, private to self
```

Practice Set — Python-Specific Details

Try these yourself first — no code shown, that's the point

1. Add __str__ to a class you've already written this session, and confirm print() uses it

2. Add a @classmethod alternate constructor to your Employee class from earlier that builds one from a single "name-salary" string

3. Deliberately reproduce the mutable class-attribute bug, confirm the leak, then fix it



INDUSTRY-STYLE TASKS

Build the Things You'll Actually Be Asked to Build

Not toy examples — validated accounts, class hierarchies, and alternate constructors are everyday, real-world work

Today's Tasks — Part A: Classes & Encapsulation

Small tasks — do them in order

- Build a BankAccount class with a protected _balance, plus deposit() and withdraw() methods that reject invalid amounts — no other way to change the balance.
- Turn balance into a @property with a setter that raises a ValueError for negative values, and confirm direct assignment is properly validated.
- Add a class attribute bank_name shared by every account, and confirm it updates for all existing accounts when changed on the class.

Today's Tasks — Part B: Inheritance & Polymorphism

Small tasks — do them in order

- Build a Shape base class (abstract, using ABC) with an abstract area() method, then Rectangle and Triangle subclasses that implement it differently.
- Loop over a list of mixed shapes and print each one’s area — without checking what type each shape is.
- Create a SavingsAccount(BankAccount) subclass that overrides withdraw() to enforce a minimum balance, using super() to reuse the original validation first.

Today's Tasks — Part C: Python-Specific Details

Small tasks — do them in order

- Add __str__ and __repr__ to your BankAccount class, with __str__ showing a friendly summary and __repr__ showing exact constructor-style detail.
- Add a @classmethod from_dict(cls, data) that builds a BankAccount from a dictionary like {"owner": "Ahmed", "balance": 500}.
- Add a @staticmethod is_valid_owner_name(name) that checks a name is non-empty and alphabetic, with no reference to self or cls.



PART G

Put It All Together

Two projects combining classes, all four pillars, and Python-specific OOP details into one cohesive task

Mini Project 1: Bank Account System



Encapsulation + inheritance, combined

- Build a BankAccount base class with a protected _balance, a validated @property, and deposit()/withdraw() methods
- Build SavingsAccount and CheckingAccount subclasses that each override withdraw() with their own rules (minimum balance, overdraft limit)
- Add __str__ for a friendly account summary, and a @classmethod that builds an account from a dictionary of starting data
- This mirrors how real banking and fintech backends structure account logic

Mini Project 2: Shape & Employee Hierarchies



Abstraction + polymorphism, combined

- Build an abstract Shape class (area(), perimeter()) with Circle, Rectangle, and Triangle subclasses
- Write a report() function that loops over a mixed list of shapes and prints each one's area — using polymorphism, with no type-checking
- Separately, build an Employee base class and a Manager subclass that overrides a bonus() calculation using super()
- This mirrors how real HR systems and geometry/CAD tools are actually structured



RECAP

Classes bundle data with behavior. The four pillars — encapsulation, abstraction, inheritance, and polymorphism — are how that bundling stays clean as programs grow.

Next session: File I/O & Error Handling — reading and writing files, try/except